ENTERPRISE MIGRATION INTELLIGENCE OPERATING SYSTEM

EMIOS

Project Documentation — Architecture, Agent Workflows, UI Reference & Calculation Methodology

Digital Twin Graph | 12-Agent LangGraph Pipeline | Monte Carlo Risk Simulation | Wave-Based Migration Planning | RAG Executive Copilot

---

Prepared: August 2026

Repository: CybHackathon-2026 / CDAI_Cortex_Creators (branch: feature/end-to-end-assessment-platform)

Scope: Full-stack review of backend agent pipeline, quantitative models, and the React frontend.

Contents

1. Executive Summary — What EMIOS Is and Why It's Useful 3
2. Technology Stack 4
3. End-to-End System Workflow 5
4. The 12-Agent Master Orchestrator Pipeline 6
5. Quantitative Calculations Reference (Cost, Days, Risk, Scores) 12
6. User Interface Reference — Pages & Components 18
7. Why EMIOS Is Useful — Business Value 24
8. Known Limitations & Engineering Notes 25

Page numbers are approximate — actual pagination depends on print rendering.

SECTION 1

Executive Summary

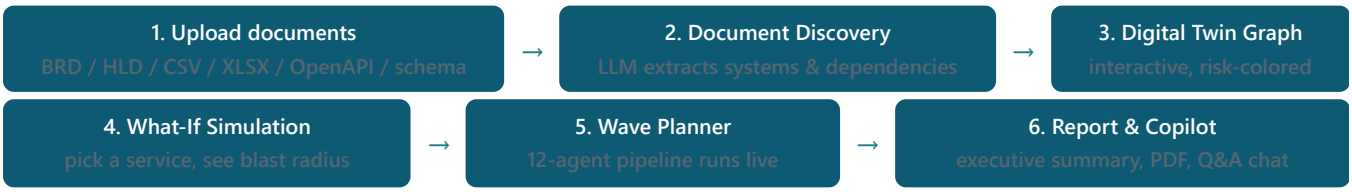
EMIOS (Enterprise Migration Intelligence Operating System) is a decision-support platform that helps an enterprise understand — before it spends a single dollar on a cloud migration — exactly what it owns, how those systems depend on each other, what breaks if a given system fails or moves, how much the migration will realistically cost and take, and in what order systems should be migrated to minimize risk.

It does this by turning a company's own uploaded documents (Business Requirement Documents, High-Level Design docs, architecture diagrams, spreadsheet inventories, OpenAPI specs, database schema exports — whatever already exists) into a live, queryable **digital twin graph** of the application landscape, then running that graph through a **12-agent AI pipeline** that assesses complexity, risk, and cloud readiness; simulates three different migration strategies with Monte Carlo cost/time forecasting; and negotiates a realistic wave-by-wave migration roadmap. All of this is exposed through a real product UI — not a slide deck — with live streaming agent output, interactive graph visualization, and a document-grounded executive chat assistant.

The core idea in one sentence

> **Most migration failures are caused by unknown dependencies, not bad code.** EMIOS exists to surface those dependencies and their blast radius *before* a migration starts, using the company's own documents as ground truth rather than industry rules of thumb.

What a user actually does



Who this is for

Audience	What EMIOS gives them
CIO / migration sponsor	A readiness score, a recommended cloud platform, and a defensible cost/time estimate — grounded in the company's actual systems, not analyst benchmarks.
Enterprise / solution architects	An accurate, auto-generated dependency graph and a wave sequence that respects real technical dependencies (nothing migrates before what it depends on).
Migration engineering teams	Per-service complexity, effort (person-days), team-size, and timeline estimates, plus circular-dependency detection with concrete decoupling strategies (Saga pattern, event broker, API gateway, etc).
Risk / compliance stakeholders	An 8-category enterprise risk score (technical, business, compliance, security, operational, performance, legacy, data-migration) with named top risks and mitigations.
Finance	Current hosting spend, projected savings, Monte-Carlo cost ranges (not single-point guesses) with an 80% confidence band.

SECTION 2

Technology Stack

Layer	Choice	Why it matters
Backend	FastAPI + Python 3.11+, Uvicorn	Async-first API serving both REST endpoints and the built frontend from one origin (port 8000)
Agent orchestration	LangGraph + LangChain	Models the 12-agent pipeline as an explicit state graph with retries and conditional routing
Graph algorithms	NetworkX	Cycle detection, topological sort (wave sequencing), BFS traversal (blast-radius simulation)
Digital twin store	Neo4j	Native graph database for the application/dependency topology
Relational store	PostgreSQL (SQLAlchemy 2.0 async) — SQLite in tests	Assessments, uploads, waves, agent runs, reports
Vector store (RAG)	Qdrant	Per-assessment document-chunk embeddings for the Executive Copilot and Discovery tool-use
Object storage	Amazon S3 (local-disk fallback)	Stores uploaded source documents
LLM + embeddings	AWS Bedrock (qwen.qwen3-coder-next default) → OpenAI/Gemini/Azure OpenAI fallback → deterministic non-AI fallback	The whole app runs and is fully testable even with zero API keys configured
Observability	Langfuse (optional)	Traces + prompt management for every LLM call
Frontend	Vite + React 19 + TypeScript + Tailwind v4 + Radix UI	Built to `frontend/dist`, served by FastAPI
Frontend extras	@xyflow/react (graph canvas), Recharts, Mermaid, react-markdown, framer-motion	Interactive graph, charts, agent-generated flowcharts, smooth panel animation
Containers	Podman or Docker (engine-agnostic compose files)	Local dev + deployment (validated on AWS Lightsail)

Graceful degradation, by design

Every infrastructure dependency has a fallback: no Neo4j → in-memory graph; no Postgres → SQLite; no Qdrant → in-memory vector search; no AWS/OpenAI/Gemini credentials → deterministic, non-AI narrative text and a hash-based pseudo-embedder. The application can run end-to-end with **zero containers and zero API keys**, which is what makes the 193-test backend suite runnable without any live infrastructure.

SECTION 3

End-to-End System Workflow

There are two distinct API surfaces: a legacy, single-tenant `/api/*` surface (health, upload, graph, simulate, plan, chat — unwrapped responses), and the durable, multi-user `/api/v1/*` surface (consistent `{success, message, data}` envelope) that the real product UI drives. Everything below describes the `/api/v1` path.

Step 1 — Document upload (RAG ingestion)

`POST /assessments/{id}/uploads` (or the `/uploads/zip` / `/uploads/zip/stream` variants for archives) chunks and embeds whatever text can be extracted from each uploaded file into Qdrant, scoped to that assessment. This endpoint is deliberately *not* a structured topology parser — it feeds the RAG retrieval used by Document Discovery's tool-use and the Executive Copilot.

Step 2 — Digital twin graph construction

Two ways to get a graph:

- **Document Discovery** (`POST /assessments/{id}/discover` or the streamed `/discover/stream` SSE variant): an LLM extraction pass reads the uploaded free-text documents and extracts systems and their dependencies, including a cost/revenue estimation step and an entity-reconciliation pass that de-duplicates the same system mentioned across multiple documents. This is what auto-builds the graph behind the scenes when a user just uploads BRDs/HLDs and clicks "Start assessment."
- **Direct structured POST** (`POST /assessments/{id}/graph`): a caller (or the CSV/JSON seeders under `Meridian_Sample_Graph/`) posts nodes/edges directly, skipping extraction entirely — useful for pre-built inventories or quick demos.

Step 3 — What-If Simulation (on demand, cheap, deterministic)

A user picks one target service; the backend runs a breadth-first cascading-failure traversal plus a Monte Carlo triangular-distribution forecast (see Section 5) — **not an LLM call** — and returns blast radius, revenue at risk, and a cost/duration confidence range. This is designed to be fast enough to re-run interactively for every candidate migration target.

Step 4 — Wave Planner (the 12-agent pipeline)

`POST /assessments/{id}/agent-runs/stream` runs the full Master Orchestrator pipeline (Section 4) over Server-Sent Events, streaming each agent's narrative and structured output live to a persistent side panel the moment it completes. The pipeline auto-starts the instant a graph exists, and a `PlannerRunProvider` mounted at the app root keeps it running (and pushes a completion toast) no matter what page the user navigates to.

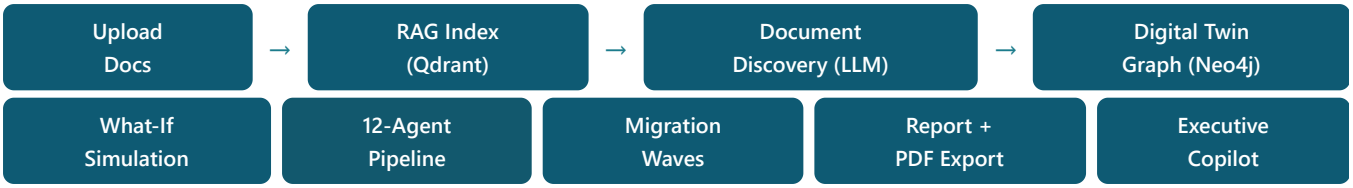
Step 5 — Report generation (auto-chained)

The moment the planner pipeline finishes, report generation starts automatically (the same provider-chaining pattern as step 4) via `POST /assessments/{id}/report` , producing an executive summary, a 0–100 readiness score, and stakeholder-framed recommendations — downloadable as PDF, with a thumbs up/down + comment feedback loop that triggers an AI revision of just the affected report section, and full version history.

Step 6 — Executive Copilot (ongoing)

A chat widget answers free-form questions grounded in *this assessment's* uploaded documents (RAG retrieval via Qdrant) plus its digital twin / report / simulation data — not a generic chatbot with no context of the customer's actual environment.

Full pipeline diagram


Upload Docs
RAG Index (Qdrant)
Document Discovery (LLM)
Digital Twin Graph (Neo4j)
What-If Simulation
12-Agent Pipeline
Migration Waves
Report + PDF Export
Executive Copilot

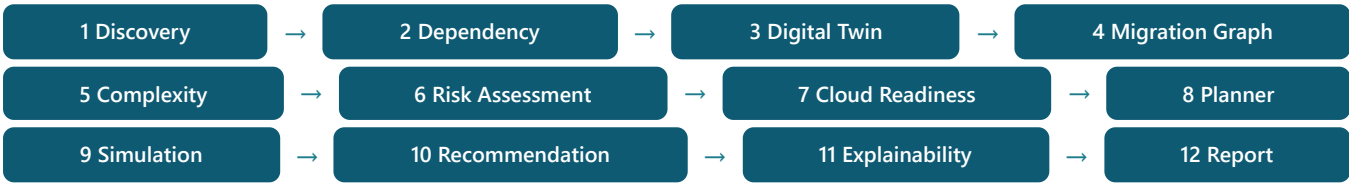
SECTION 4

The 12-Agent Master Orchestrator Pipeline

This is the heart of EMIOS: a LangGraph state machine (`run_full_assessment_stream` in `backend/app/agents/workflow.py`) that runs twelve specialized agents in a strict linear sequence over one shared state object, streaming each agent's output live to the UI as it completes.

> **Governing convention — deterministic data, LLM narrative only.** Every agent computes its real structured data (scores, tables, wave sequences) in plain Python *before* calling the LLM. The LLM call supplies only a short (2–5 sentence) plain-English narrative interpretation — every agent's system prompt explicitly forbids the model from producing Markdown, JSON, or bullet points. A separate deterministic rendering layer (`response_formatter.py`) combines the narrative with the exact Python-computed numbers to build the tables, bullet lists, and Mermaid diagrams actually shown in the UI. **Practical implication:** if a number on screen is ever wrong, the bug is in the Python computation or the formatter — never the prompt.

Pipeline order



AGENT 1 — DISCOVERY AGENT

Computes: builds the set of every service referenced by a dependency edge; anything *not* in that set is flagged as an **orphan** (no declared connections — a data-quality / hidden-dependency risk flag). Groups dependencies by database to find **shared databases** (accessed by more than one client — a coupling risk).

LLM role: explains why the orphans/shared-DB findings matter. Uniquely agentic — it can call a `search_schema_graph` tool to look up a specific system or uploaded-document fact instead of guessing.

UI shows: narrative + "Orphaned services" bullet list + shared-database bullets.

AGENT 2 — DEPENDENCY ANALYSIS AGENT

Computes: runs NetworkX `simple_cycles()` over the dependency graph to find every circular dependency loop, then synthesizes a decoupling strategy per cycle (Saga Pattern for distributed transactional loops, Event-Driven Broker for async cycles, API Gateway / Interface Abstraction for tight synchronous coupling, Database Decoupling for shared-DB access).

LLM role: narrates why each detected cycle matters for migration sequencing.

UI shows: narrative + a Mermaid closed-loop diagram per cycle (e.g. `A → B → C → A`) + the recommended decoupling strategy.

AGENT 3 — ENTERPRISE DIGITAL TWIN AGENT

Computes: converts the raw service/dependency lists into React Flow node/edge shapes for the graph canvas, and tallies a node-type breakdown (how many Microservices, Databases, Queues, etc).

LLM role: summarizes structural highlights — hub systems, isolated clusters, database-heavy zones.

UI shows: narrative + a "Type / Count" table.

AGENT 4 — MIGRATION INTELLIGENCE GRAPH AGENT

Computes: classifies every service into Applications vs. Databases, extracts Queue/ETL edges (MQ/Async type), flags externally-discovered systems, and folds in the Dependency Agent's decoupling strategies.

LLM role: describes how applications/databases/queues/external systems interconnect for planning purposes.

UI shows: narrative + an "Entity type / Count" table.

AGENT 5 — COMPLEXITY ASSESSMENT AGENT

Computes: a 5-dimension weighted complexity score (Architecture, Business, Technology, Integration, Migration — full formulas in Section 5.4), then derives engineering effort in person-days, required team size, and an estimated timeline in weeks.

LLM role: interprets the five dimension scores and the derived effort/timeline/team-size numbers for a stakeholder audience.

UI shows: narrative + a "Dimension / Score" percentage table + a bold estimated-effort summary line.

AGENT 6 — RISK ASSESSMENT AGENT (ENTERPRISE / PORTFOLIO-LEVEL)

Computes: 8 category risk scores — technical, business, compliance, security, operational, performance, legacy, data-migration (full formulas in Section 5.5) — averaged into one overall risk score, plus a list of named top risks and matching mitigation strategies.

LLM role: narrates the 8-category risk picture and why the top risks matter.

UI shows: narrative + "Risk category / Score" table (all 8) + "Top risks" bullets + "Mitigations" bullets.

AGENT 7 — CLOUD READINESS AGENT

Computes: an overall readiness score from complexity + risk + business value (Section 5.6), then keyword-matches each service's runtime string against Azure / AWS / GoogleCloud technology signatures to score platform fit and recommend one.

LLM role: explains the readiness score across the three platforms. The prompt encodes an explicit business preference for AWS when technical compatibility is roughly equal (matching EMIOS's own AWS Bedrock/S3/Lightsail deployment stack) — a deliberate business bias layered on top of, not baked into, the numeric score.

UI shows: narrative + "Platform / Readiness" table with the recommended platform highlighted.

AGENT 8 — MIGRATION PLANNER AGENT

Computes: a topological sort (Kahn's algorithm) over the dependency graph — systems with no unresolved dependencies form Wave 1, then each subsequent wave is whatever becomes unblocked once the prior wave is "migrated." Any system still part of an unbroken cycle is appended to the final wave. One hardcoded heuristic can move a monolith out of Wave 1 into Wave 2 if risk objections warrant it (see Section 5.7).

LLM role: explains the wave sequencing choices — keeping high-revenue services in later/stable waves, grouping tightly-coupled or shared-DB services together, keeping foundational systems (auth/infra) in Wave 1.

UI shows: narrative + a Mermaid left-to-right flowchart with each wave as its own subgraph (or a plain Wave/Services table for a single wave) + any adjustments made.

AGENT 9 — SIMULATION AGENT

Computes: runs the real Monte Carlo engine three times — once per fixed migration strategy (Lift & Shift, Replatform, Refactor), each with its own cost/duration/risk multiplier — to produce a comparable cost, duration, effort, business-impact, and confidence figure per strategy (full formulas in Section 5.8).

LLM role: compares the three strategies' tradeoffs for both business and engineering audiences.

UI shows: narrative + a "Strategy / Risk / Duration / Cost / Confidence" comparison table across all three.

AGENT 10 — RECOMMENDATION AGENT

Computes: a weighted composite score per strategy (50% risk, 25% relative duration, 25% relative cost — see Section 5.9) and picks the lowest-composite strategy as the recommendation, with a confidence score blending that strategy's Monte Carlo confidence and the cloud-readiness score.

LLM role: deliberately constrained — the strategy and confidence are already decided by the formula; the model's only job is to explain *why* that specific choice is correct, even reconciling its own read of the data in favor of the chosen strategy rather than proposing an alternative.

UI shows: narrative + a bold "Recommended strategy: X (NN% confidence)" headline + four stakeholder-framed recommendation blocks: Engineering, Business, Modernization, Customer-facing.

AGENT 11 — EXPLAINABILITY AGENT

Computes: pure traceability over already-produced state — no new numbers. Pulls the cycle paths and descriptions behind each decoupling strategy as "evidence," copies the top risks list, and writes a templated confidence-explanation sentence citing the recommendation's confidence score, the enterprise risk score, and the complexity score.

LLM role: explains why the recommendation was generated, citing only evidence already produced by earlier agents — the prompt explicitly forbids introducing new facts.

UI shows: narrative + "Evidence considered" bullets + "Risks considered" bullets + the confidence-explanation sentence.

AGENT 12 — REPORT GENERATION AGENT

Computes: an assessment summary (total services, total dependencies, wave count) and assembles every prior agent's structured output — complexity, enterprise risk, cloud readiness, simulation results, recommendation — into one JSON bundle for the report. Performs no new arithmetic: the report's displayed readiness score *is* the Cloud Readiness Agent's score verbatim, and its confidence *is* the Recommendation Agent's confidence verbatim.

LLM role: writes a concise executive summary tying readiness, complexity, risk, waves, simulation results, and the recommendation together for a non-technical audience.

UI shows: narrative + "N services and M dependencies assessed across K migration wave(s)" + the recommended strategy headline.

Live streaming (SSE) and reliability

The pipeline is invoked with LangGraph's `stream_mode="values"`, which yields the full accumulated state after every node completes; the streaming wrapper diffs the running narrative log against what's already been sent and streams only the new entries — one event per completed agent — over Server-Sent Events. Each event is persisted to the database immediately, so a run survives even if the browser tab is closed mid-pipeline. Every one of the 12 nodes is wrapped in a retry handler: on failure it retries once (2 attempts total); if it still fails, the whole pipeline halts and flags `HUMAN_REVIEW_REQUIRED` rather than continuing on corrupted state or silently defaulting a score to zero.

Legacy 4-agent negotiation loop

An older, still-functional code path (`run_agent_negotiation`) exists alongside the 12-agent pipeline, reusing the same Discovery, Dependency, and Planner functions but pairing Planner with a simpler per-service **wave-sequencing risk agent** (not the 8-category enterprise risk agent) in an actual back-and-forth loop: Planner proposes waves, Risk raises objections if a service is scheduled ahead of something it depends on, and Planner revises — up to 5 rounds — until Risk is satisfied or the cap is hit. This is what powers the legacy `/api/plan` endpoint; the real product UI's Wave Planner page uses the strictly linear 12-agent pipeline instead, which does not loop.

SECTION 5

Quantitative Calculations Reference

This section documents, formula by formula, every number EMIOS shows: cost, days/duration, risk percentages, complexity scores, readiness scores, and confidence. All formulas below are taken directly from `backend/app/core/constants.py`, `backend/app/services/simulation_engine.py`, and `backend/app/agents/workflow.py`.

5.1 Cascading-Failure Blast Radius (BFS)

When a user runs What-If Simulation on a target service, EMIOS reverses every dependency edge (so a breadth-first walk visits "who breaks next," not "what this depends on") and starts the target at 100% disruption. For each hop to a downstream client:

```
crit_factor = 0.8 (High criticality) | 0.4 (Medium) | 0.1 (Low)
type_mod    = 1.2 (Sync call)     | 1.1 (DB access) | 0.5 (Async / Message Queue)

new_risk = current_risk × crit_factor × type_mod   # clamped to [0, 1]
```

A client's risk is only propagated further if the resulting value exceeds a 5% noise floor *and* is higher than any risk already recorded for that node from another path (max-risk relaxation). Because `crit_factor` never exceeds 0.8, risk decays multiplicatively with every hop unless amplified by a Sync or DB type modifier — this is why it behaves like a real shockwave: a fast-decaying wave of disruption probability radiating outward from the migrated system, not a flat "everything downstream breaks equally" model.

REVENUE AT RISK

```
revenue_at_risk = target.monthly_revenue_impact              # 100%, it's being migrated
                + Σ (impacted_node.monthly_revenue_impact × impacted_node.risk)  # probability-
weighted
```

In plain terms: the expected dollar value of monthly revenue exposed to disruption, summing the target's full revenue plus every downstream service's revenue contribution weighted by how likely that service is to be disrupted.

5.2 Monte Carlo Cost & Duration Simulation

Rather than a single-point cost/time guess, EMIOS runs 1,000 simulated outcomes per request using a **triangular distribution** (minimum / most-likely / maximum) — the standard three-point estimation technique used in project management (PERT), chosen specifically because it needs no calibrated variance parameter and naturally models the business intuition that overruns are more likely than underruns.

```
# per node, per simulated run — duration in days
min_duration  = base_duration × 0.9
mode_duration = base_duration × (1.0 + risk × 0.2)
max_duration  = base_duration × (1.1 + risk × 2.0)

# per node, per simulated run — cost in USD
min_cost  = base_cost × 0.95
mode_cost = base_cost × (1.0 + risk × 0.3)
max_cost  = base_cost × (1.2 + risk × 2.5)

sampled_duration = random.triangular(min_duration, mode_duration, max_duration)
```

```
sampled_cost     = random.triangular(min_cost,     mode_cost,     max_cost)
# summed across every node in the topology → one run's total; repeated 1,000×
```

Across all 1,000 runs, EMIOS reports:

- **Expected duration / cost** — the mean across all simulated runs
- **90% confidence range** — the 10th-to-90th percentile band (an 80%-width interval despite the "90%" name — it's the upper percentile bound, not the interval width)

At maximum risk (risk = 1.0), a service's worst-case simulated duration can reach **3.1×** its base estimate and worst-case cost **3.7×** — reflecting how strongly failure risk inflates the tail of the distribution, not just its center.

5.3 Migration Cost, Duration & Revenue Estimators (at ingestion time)

When a system's annual hosting cost is known (from an uploaded JSON inventory), EMIOS derives its estimated migration cost, duration bucket, and monthly revenue impact automatically:

```
complexity_tier = "High"    if annual_hosting_cost > $100,000
                = "Medium"  if annual_hosting_cost > $20,000
                = "Low"     otherwise

migration_cost     = max(annual_hosting_cost × 1.2, $5,000)          # floored at $5,000
migration_duration = 90 days (High) | 30 days (Medium) | 14 days (Low)
monthly_revenue    = (annual_hosting_cost × 10.0) / 12               # implicit: revenue ≈ 10×
hosting cost
```

When a CSV inventory is uploaded instead, these figures are read directly from human-supplied columns (e.g. "Migration Cost ($)", "Migration Time (Days)", "Annual Hosting Cost ($)") rather than estimated.

TOTAL HOSTING COST & POTENTIAL SAVINGS (PORTFOLIO-WIDE)

```
total_hosting_cost = Σ every service's annual hosting cost
potential_savings  = total_hosting_cost × 0.30        # flat assumption: cloud migration cuts
hosting spend 30%
```

5.4 Complexity Score

Five dimensions, each normalized to 0–1, then combined with fixed weights that sum to exactly 1.00:

Dimension	Weight	What it measures
Architecture	25%	Average fan-out (dependencies per service) plus a penalty per circular dependency detected
Business	15%	Share of services flagged High business value
Technology	20%	Runtime/technology diversity across the portfolio
Integration	25%	Share of dependency edges that are High-criticality or synchronous
Migration	15%	Average per-service migration-complexity rating (High/Medium/Low)

```
complexity_score = 0.25×architecture + 0.15×business + 0.20×technology
                 + 0.25×integration  + 0.15×migration        # result: 0.00–1.00

engineering_effort_days  = total_base_duration × (0.5 + complexity_score)
team_size                = max(2, round(engineering_effort_days / 90))
```

```
estimated_timeline_weeks = max(1, round(engineering_effort_days / team_size / 5))   # 5
workdays/week
```

In plain terms: a portfolio with lots of tightly-coupled, high-value, high-complexity, diverse-technology systems scores close to 1.0 — meaning more person-days of effort, a larger recommended team, and a longer calendar timeline for the same total base-duration estimate.

5.5 Enterprise Risk Score

Eight category scores (0–1 each), simple unweighted average:

Category	Formula
Technical	Average migration-complexity weighting (High=1.0, Medium=0.5, Low=0.2) across services
Business	Share of services with High business value
Compliance	Share of services that are databases
Security	Share of services with zero declared dependencies (orphans)
Operational	Circular-dependency count × 0.2 (capped at 1.0)
Performance	Share of edges that are both High-criticality and synchronous
Legacy	Share of services flagged status = "Legacy"
Data migration	Share of dependency edges that are database-type

```
enterprise_risk_score = average(all 8 category scores)     # 0.00–1.00
```

5.6 Cloud Readiness Score & Platform Fit

```
readiness_score = clamp(
    1.0
    − complexity_score    × 0.40
    − enterprise_risk      × 0.35
    + business_value_share × 0.25,
    0.0, 1.0
)
```

Readiness starts at a perfect 1.0 and is penalized by complexity and risk, boosted by the share of high-value services. Below a **0.60** threshold, EMIOS explicitly recommends modernizing before migrating rather than lifting legacy problems straight into the cloud.

```
platform_score[p] = 0.5 × readiness_score
                  + 0.5 × (platform_p_keyword_matches / max_platform_matches)
recommended_platform = the platform (AWS / Azure / GoogleCloud) with the highest platform_score
```

Platform fit is a 50/50 blend of overall readiness and how many services' declared runtime strings match that platform's technology keywords (e.g. .NET/SQL Server/Windows → Azure signal; Java/Node/Postgres/MySQL/DynamoDB → AWS signal; Python/Kubernetes/BigQuery/TensorFlow → Google Cloud signal).

5.7 Migration Wave Sequencing

Wave assignment is **pure dependency topology**, computed via a topological sort (Kahn's algorithm) — not a weighted score:

```
Wave 1 = every service with zero unresolved dependencies
Wave 2 = services that become unblocked once Wave 1 is "migrated"
Wave N = repeat until every service is placed
(any service still stuck in an unbroken cycle is appended to the LAST wave)
```

A companion **wave-sequencing risk check** (used in the legacy 4-agent negotiation loop) scores each proposed placement:

```
violation += 0.40   if a High-criticality dependency is scheduled in a LATER wave than its consumer
violation += 0.20   if a Medium-criticality dependency is scheduled later
violation += 0.15   if a High-criticality, synchronous dependency shares the SAME wave (tight-coupling penalty)

final_risk = min(violation × complexity_multiplier, 1.0)     # multiplier: 1.3 High / 1.1 Medium / 1.0 Low
→ objection raised if final_risk > 0.15
```

Objections feed back to the Planner, which currently applies one concrete fix: deferring a monolith out of Wave 1 into Wave 2 so its database/auth dependencies can go first.

5.8 What-If Simulation: Three Migration Strategies

Strategy	Cost multiplier	Duration multiplier	Risk multiplier	Profile
Lift & Shift	0.7×	0.6×	1.3×	Cheapest and fastest, but riskiest — minimal re-architecture
Replatform	1.0×	1.0×	1.0×	Baseline — moderate changes to fit the cloud platform
Refactor	1.6×	1.8×	0.7×	Most expensive and slowest, but lowest risk — real re-architecture

For each strategy, EMIOS scales every service's cost/duration by that strategy's multipliers, scales the overall enterprise risk by the risk multiplier, and re-runs the full Monte Carlo engine (Section 5.2) on the scaled topology — producing genuinely different cost/duration distributions per strategy, not just a labeled guess.

5.9 Recommended Strategy & Confidence

```
composite(strategy) = risk × 0.50
                    + (duration / max_duration_across_strategies) × 0.25
                    + (cost / max_cost_across_strategies) × 0.25

recommended_strategy = the strategy with the LOWEST composite score
confidence_score     = (chosen_strategy.confidence + cloud_readiness_score) / 2
```

Risk dominates the recommendation (50% of the composite weight), with cost and duration splitting the remainder evenly — reflecting a "don't be reckless" bias baked into which strategy gets recommended by default.

5.10 Quick Reference — All Tunable Constants

Constant	Value	Governs
Criticality factor High / Medium / Low	0.8 / 0.4 / 0.1	Blast-radius attenuation per hop
Type modifier Sync / DB / Async-MQ	1.2 / 1.1 / 0.5	Blast-radius attenuation per edge type
Monte Carlo duration min/max factors	0.9× / (1.1 + risk×2.0)×	Triangular distribution bounds — duration
Monte Carlo cost min/max factors	0.95× / (1.2 + risk×2.5)×	Triangular distribution bounds — cost
Planner max iterations	5	Cap on Planner↔Risk revision rounds (legacy loop)
Planner risk objection threshold	0.15	Wave-placement risk above this raises an objection
Migration cost factor / floor	1.2× / $5,000	Estimated migration cost from hosting cost
Revenue multiplier	10.0×	Estimated annual revenue from hosting cost
Migration duration buckets	14 / 30 / 90 days	Low / Medium / High complexity tiers
Cloud savings multiplier	30%	Potential savings vs. current hosting cost
Complexity dimension weights	25/15/20/25/15%	Architecture/Business/Technology/Integration/Migration
Cloud readiness weights	−40% complexity, −35% risk, +25% business value	Readiness score composition
Simulation strategy multipliers	see Section 5.8	Lift&Shift / Replatform / Refactor scaling
Modernization threshold	0.60 readiness	Below this, EMIOS recommends modernizing before migrating
Force-all-strategies threshold	0.80 enterprise risk	Informational flag when portfolio risk is very high
Agent retry budget	1 retry (2 attempts total)	Before a pipeline stage halts and requests human review
Monte Carlo simulation runs	1,000	Sample size for every cost/duration forecast

All of the above live in `backend/app/core/constants.py` by project convention (new tunable numbers belong there, not inline in business logic) — a handful of secondary coefficients (e.g. the Monte Carlo mode-skew terms, a few keyword-match lists, the composite-recommendation weights) remain inline in `workflow.py` / `simulation_engine.py` as of this writing.

SECTION 6

User Interface Reference — Pages & Components

The frontend is a single-page React application with one persistent left sidebar (`AppShell`) and a route per screen. Three background-run providers (Discovery, Planner, Report) are mounted once at the app root, each with its own small floating status panel stacked bottom-right, so long-running work keeps going — and notifies the user on completion — no matter what page they navigate to.

Route	Page	Purpose
`/`	Home	Public marketing landing page
`/login`	Login	Login / register (JWT session cookie)
`/dashboard`	Dashboard	Portfolio view of all the user's assessments
`/assessments/new`	New Assessment	Create an assessment + upload documents
`/assessments/:id`	Overview	Per-assessment command center — KPIs, charts, recommendations
`/assessments/:id/documents`	Documents	Upload + run Document Discovery
`/assessments/:id/graph`	Digital Twin Graph	Interactive risk-colored dependency graph
`/assessments/:id/simulate`	What-If Simulation	Blast radius + Monte Carlo forecast for one service
`/assessments/:id/planner`	Wave Planner	Runs and explains the 12-agent pipeline live
`/assessments/:id/report`	Report	Executive summary, readiness score, feedback, PDF export
`/settings`	Settings	Account info + logout

6.1 Dashboard

Three stat tiles — **Total assessments**, **In progress**, **Completed** — plus a grid of assessment cards (project name, customer, target cloud, colored status badge: created/uploading/analyzing/complete/failed).

6.2 New Assessment

Two-column form: customer name / project name / target cloud on the left with a drag-drop file picker (PDF, DOCX, PPTX, XLSX, CSV, JSON, TXT, MD, or ZIP); a live review panel on the right showing staged/uploaded files with per-file preview and status. "Upload N documents" then "Start assessment" triggers Document Discovery in the background and lands the user on the Overview page.

6.3 Overview Page (the command center)

0–100
MIGRATION READINESS — FROM THE LATEST REPORT

$
TOTAL ANNUAL HOSTING COST — REAL CURRENT SPEND, SUMMED ACROSS SERVICES

%
OVERALL RISK SCORE — FROM THE LATEST PLANNER RUN'S ENTERPRISE RISK SCORE

%
MIGRATION COMPLEXITY — FROM THE LATEST PLANNER RUN'S COMPLEXITY SCORE

- **Score Ring** — an animated circular gauge (teal→red gradient) showing the readiness score, spinning while the report is still generating.
- **Pipeline metric cards** — full detail breakdowns of Complexity, Enterprise Risk, and Cloud Readiness (see Section 6.6 for their shared building blocks).

- **Migration wave planner bar chart** — one bar per wave, height = number of services in that wave.
- **Risk distribution donut chart** — buckets every graph node into Low/Medium/High by its `failure_probability` (High ≥0.6, Medium ≥0.25, else Low) — the same thresholds used to color the graph page, so the two views always agree.
- **Service inventory table** — name, type, business value, annual cost, one row per system.
- **AI recommendations panel** — the report's bullet-list recommendations.

6.4 Documents Page

Upload dropzone; a list of uploaded documents with filename, source type, chunk count (how many text chunks it was split into for RAG), and status. A "Process Documents" button starts Document Discovery, streamed live as a scrolling log (e.g. `"[2/5] Processing spec.pdf ..."`, `"→ extractor found 4 systems and 3 dependencies (confidence 82%)"`). On completion: node/edge counts, CSV downloads, and a link to the graph.

6.5 Digital Twin Graph Page

An interactive, auto-laid-out (Dagre hierarchical layout) node-and-edge canvas built on `@xyflow/react`.

Low risk — failure_probability < 0.25

Medium risk — 0.25 ≤ failure_probability < 0.6

High risk — failure_probability ≥ 0.6

Hovering or selecting a node highlights it and its directly connected neighbors, dimming everything else. Edges carry their own independent color from the `criticality` field (High/Medium/Low), separate from node risk. Selecting a node opens a detail card: business value, migration complexity, annual hosting cost, base migration cost, base duration (days), runtime, and upstream/downstream dependency chip lists. CSV export (`nodes.csv` / `edges.csv`) is available directly from this page.

6.6 What-If Simulation Page

Input: pick one target service from a dropdown, click "Simulate migration risk."

Output:

$
CURRENT HOSTING COST — OF THE SIMULATED TOPOLOGY

$
POTENTIAL SAVINGS — PROJECTED IF MIGRATION PROCEEDS (30% OF HOSTING COST)

$/mo
REVENUE AT RISK — IF THE TARGET FAILS MID-MIGRATION

Plus a plain-English summary sentence, a Monte Carlo best/expected/worst-case range bar for both duration and cost (90% confidence band), and a mini risk-propagation graph showing the actual cascading blast-radius path animated in red. A separate "strategy simulation" card shows the Lift&Shift / Replatform / Refactor comparison from the 12-agent pipeline's Simulation Agent (Section 5.8) on the same page.

6.7 Wave Planner Page

A "Run planner" button streams the full 12-agent pipeline live. Sections: the proposed migration waves (numbered, with resolved service names), overall pipeline confidence, an Explainability card (evidence + dependencies/risks considered), and one detailed card per agent in the run — including retry-attempt badges and a "Fallback response" badge when no LLM provider was configured for that stage (so the difference between a real model narrative and the deterministic canned text is always visible to the user, never silently hidden).

6.8 Report Page

1. **Executive summary** — markdown-rendered narrative, a "Readiness NN/100" badge, and bullet recommendations.
2. **Final migration plan** — a generated go/no-go strategy document, downloadable as its own PDF.
3. **Version history** — every past revision, expandable, with source (regenerated vs. feedback-revised), timestamp, and changed sections.
4. **Feedback** — thumbs up ("Looks good") or thumbs down ("Request changes") with a free-text comment; a thumbs-down triggers an AI revision that rewrites only the report section(s) the feedback concerns (Executive summary / Risk summary / Migration waves / Recommendations), leaving the rest untouched.

6.9 Executive Copilot

A floating chat panel (bottom-left) titled "Executive Decision Copilot." Grounded in the current assessment's uploaded documents (RAG retrieval) plus a live summary of its digital twin graph, report, and simulation data — so answers reference the customer's actual systems, not generic advice. If the backend call fails, it degrades to a local keyword-matching responder using whatever context is already loaded client-side, rather than going silent.

6.10 Shared Building Blocks

Component	What it does
KpiCard	Colored metric tile (icon + big value + caption); color tone chosen by the calling page based on what the metric means (cost=destructive/red, readiness=primary), not by a numeric threshold in the component itself
ScoreBar	Labeled 0–100% progress bar whose color flips meaning by direction — for risk/complexity, high = red (bad); for readiness/confidence, high = green (good)
ComplexityCard / EnterpriseRiskCard / CloudReadinessCard	Full detail views of each whole-portfolio score plus its sub-dimensions
Markdown + Mermaid renderer	Renders every agent's narrative text, intercepting fenced ```` ```mermaid ```` blocks to draw actual flowchart diagrams instead of literal code
Run panels (Discovery / Planner / Report)	Floating bottom-right status cards for background operations — spinner while running, auto-dismiss ~10s after completion, deep link to the relevant tab
Theme toggle	Light ("enterprise") theme is the default; dark ("Blackout," the original theme) is opt-in from the account menu — persisted, no flash on load

SECTION 7

Why EMIOS Is Useful

Enterprise cloud migrations fail — or run wildly over budget and schedule — for a small, repeatable set of reasons: nobody has an accurate map of what depends on what, cost/time estimates are single-point guesses instead of ranges with confidence, migration order is decided by convenience rather than dependency safety, and by the time risk becomes visible it's already an incident. EMIOS is built to attack exactly those failure modes, using the customer's own documents as the source of truth rather than industry rules of thumb.

Common migration failure mode	How EMIOS addresses it
Undocumented / forgotten dependencies surface mid-migration as outages	Document Discovery + the digital twin graph surface orphaned services and shared-database coupling *before* migration starts, not during it
Circular dependencies block a clean migration order	Automated cycle detection with a concrete, named decoupling strategy per cycle (Saga, event broker, API gateway, DB decoupling)
"It'll take 3 months" turns into 9, with no one able to explain why	Monte Carlo cost/duration ranges with an explicit 80%-confidence band, not a single optimistic number
Migration order is picked by whoever asks loudest, not by what's technically safe	Wave sequencing is a real topological sort over dependencies — nothing is scheduled before what it needs
No one can say which cloud platform actually fits the existing tech stack	Cloud Readiness scoring against Azure/AWS/GCP based on the portfolio's real declared runtimes
Risk is discussed qualitatively ("this feels risky") with no shared number	An 8-category, formula-driven Enterprise Risk score everyone is looking at the same version of
Executives don't have time to read a 40-page architecture assessment	An auto-generated executive summary + single readiness score + PDF export, with a feedback loop to correct it
Answering a stakeholder's ad-hoc question means re-reading source documents	The Executive Copilot answers directly from the assessment's own documents and computed data

What makes the numbers trustworthy rather than "AI-sounding"

The single most important design decision in EMIOS is the separation of computation from narration (Section 4): every score, table, and wave sequence shown in the product is computed by deterministic Python code using the customer's real uploaded data — the LLM is used only to explain those numbers in plain English, never to invent them. That means the same graph always produces the same complexity/risk/readiness scores, the recommendation logic is auditable line-by-line (Section 5), and the system keeps working, in a fully testable and demoable way, even with zero AI provider credentials configured.

SECTION 8

Known Limitations & Engineering Notes

- **No CI** — tests currently run manually (193 passing, no live infrastructure required).
- **RAG retrieval quality depends on configured credentials** — without real Bedrock/OpenAI/Gemini credentials, embeddings fall back to a deterministic hash-based pseudo-embedder; the pipeline runs end-to-end but retrieved chunks won't be semantically meaningful.
- **Report PDF/DOCX generation was, as of the code reviewed, only implemented for the report itself** via a separate rendering path — the 12-agent pipeline's own `report_agent` stage marks PDF/DOCX as "not implemented" in its raw output envelope; the product's actual PDF download button uses a different, working code path.
- **Human review on report feedback is intentionally simplified** — thumbs up/down + comment, no reviewer role or formal approval gate.
- **Auth authorization returns 404, not 403**, on another user's assessment — deliberate, so a caller can't distinguish "doesn't exist" from "exists but isn't yours."
- **The legacy `/api/*` surface and 4-agent negotiation loop still exist** in the codebase but are not what the live product UI drives — don't assume the legacy surface reflects current agent behavior.
- **A handful of secondary coefficients remain inline rather than centralized** in `constants.py` (e.g. Monte Carlo mode-skew terms, platform-keyword match lists, the recommendation composite weights) — functionally correct, but a minor deviation from the project's own "new tunable numbers go in constants.py" convention, worth tidying in a future pass.

> **Document generated by an automated codebase analysis** covering `backend/app/agents/`, `backend/app/core/constants.py`, `backend/app/services/simulation_engine.py`, and the full `frontend/src/` tree, cross-checked against the project's own `README.md`. Line numbers and formulas were verified directly against source at the time of writing; if the codebase changes, re-run the analysis rather than treating this document as a permanent spec.